TECHNICAL WHITEPAPER

StateSet *Desktop*

A cross-platform Electron application for managing autonomous AI customer service agents — built on enterprise-grade resilience patterns, real-time streaming, and a zero-trust security model.

VERSION	DATE	PLATFORM	LICENSE
1.1.15	March 2026	macOS · Windows · Linux	MIT

ABSTRACT

StateSet Desktop is a cross-platform Electron application that provides a native desktop interface for managing autonomous AI customer service agents. Built on a modern React/TypeScript stack with enterprise-grade resilience patterns, the application enables operators to deploy, monitor, and interact with AI agents in real time across customer support and e-commerce workflows. This paper details the system architecture, security model, data flow patterns, real-time streaming infrastructure, voice interface, and engineering decisions that underpin the platform.

CONTENTS

01 Introduction
02 System Architecture
03 Electron Main Process
04 Renderer Application
05 Data Model & Type System
06 API Client & Resilience Layer
07 Real-Time Streaming
08 State Management
09 Security Architecture
10 Offline Resilience & Caching
11 Platform Integrations & OAuth
12 Voice Interface
13 Agent Templates & Configuration
14 Observability & Diagnostics
15 Testing Strategy
16 Build & Distribution
17 Future Architecture
18 Conclusion

01 Introduction

1.1 Problem Statement

Modern customer service operations require autonomous AI agents that can handle complex workflows — processing returns, managing orders, triaging tickets — with minimal human oversight. While cloud dashboards serve this need for browser-based access, operators managing critical agent fleets require a dedicated desktop application that provides:

- Native system integration (tray agents, desktop notifications, keyboard shortcuts)
- Real-time streaming visibility into agent reasoning and tool usage
- Offline-tolerant operation for unreliable network environments
- Secure local credential storage via OS-level encryption
- Background agent monitoring without a browser tab
- Voice-based agent interaction via speech-to-text and text-to-speech
- Reusable agent templates for rapid deployment across brands

1.2 Design Goals

StateSet Desktop was designed around five core principles:

01 **Resilience over availability** — The application must degrade gracefully. Circuit breakers, request deduplication, retry with backoff, and offline caching ensure the operator experience survives transient API failures.

02 **Security by default** — Context isolation, sandboxed renderers, CSP enforcement, and OS-level credential encryption are non-negotiable. No raw IPC channels are exposed.

03 **Operator efficiency** — Command palette navigation, comprehensive keyboard shortcuts, optimistic mutations, and background agent monitoring minimize time-to-action.

04 **Observable systems** — Structured logging, request metrics with percentile tracking, circuit breaker state visibility, and audit trails make the system introspectable.

05 **Cross-platform fidelity** — A single codebase produces native-quality experiences on macOS (Intel & Apple Silicon), Windows, and Linux.

1.3 Technology Summary

LAYER	TECHNOLOGY
Runtime	Electron 35 (Chromium + Node.js)
UI Framework	React 18 + TypeScript 5.3
Build System	Vite 6.4 + electron-builder 26
State Management	Zustand 4.4 (client) + React Query 5 (server)
Styling	Tailwind CSS 3.4 + CSS Custom Properties
Validation	Zod 4.3 (runtime schema enforcement)
Animation	Framer Motion 10
Voice	ElevenLabs (STT/TTS)
Error Tracking	Sentry (main + renderer processes)
Testing	Vitest 4 (unit) + Playwright 1.58 (E2E)

1.4 Codebase Metrics

~37.5K TYPESCRIPT LOC	12 PAGES	8 FEATURE MODULES
48 SHARED COMPONENTS	27 CUSTOM HOOKS	78 TEST FILES

02

System Architecture

2.1 High-Level Topology

```
┌──────────────────────────────────────────────────────────────────────┐
│                          StateSet Desktop                            │
│                                                                      │
│  ┌──────────────────────┐          IPC          ┌──────────────────────────┐  │
│  │  Main Process        │◄──────────────────►│  Renderer Process        │  │
│  │  (Node.js)           │   (validated)      │  (Chromium Sandbox)      │  │
│  │                      │                    │                          │  │
│  │  - Window mgmt       │                    │  ┌────────────────────┐  │  │
│  │  - System tray       │                    │  │  React App         │  │  │
│  │  - Auto-updater      │                    │  │  - Pages/Routes    │  │  │
│  │  - OAuth servers     │                    │  │  - Zustand Stores  │  │  │
│  │  - Secure store      │                    │  │  - React Query     │  │  │
│  │  - CSP headers       │                    │  │  - SSE Streaming   │  │  │
│  │  - Notifications     │                    │  │  - Voice (STT/TTS  │  │  │
│  └──────────────────────┘                    │  └────────────────────┘  │  │
│                                                          │                  │
└──────────────────────────────────────────────────────────┼──────────────────┘
                                                           │
                                                           │
                                  HTTPS + SSE              │
                                                           ▼
                                  ┌──────────────────────────────┐
                                  │  StateSet Engine API         │
                                  │  engine.stateset.cloud       │
                                  │                              │
                                  │  - Agent orchestration       │
                                  │  - Session management        │
                                  │  - SSE event streaming       │
                                  │  - Webhook delivery          │
                                  │  - OAuth token exchange      │
                                  └──────────────────────────────┘
                                                 │
                                  ┌──────────────┼──────────────┐
                                  ▼              ▼              ▼
                              ┌──────────┐   ┌──────────┐   ┌──────────┐
                              │ Shopify  │   │ Gorgias  │   │ Zendesk  │
                              └──────────┘   └──────────┘   └──────────┘
```

2.2 Process Isolation Model

Electron enforces strict process separation:

Main Process — Privileged Node.js process managing windows, system tray, auto-updates, OAuth HTTP servers, and encrypted credential storage. No direct renderer access.

Preload Script — A narrow bridge exposing 30+ typed IPC methods via `contextBridge.exposeInMainWorld()`. Every method maps to a validated `ipcRenderer.invoke()` call.

Renderer Process — Sandboxed Chromium instance running the React application. No Node.js APIs, no `require()`, no filesystem access. All system interactions route through the preload bridge.

2.3 Directory Organization

The codebase follows a feature-based modular structure:

Electron Layer

```
electron/
├── main.ts              # 1,359 lines
├── preload.ts           # 30+ IPC methods
├── url-security.ts      # Navigation guards
├── sanitization.ts      # Sentry redaction
└── oauth/
    ├── shopify.ts
    ├── gorgias.ts
    ├── zendesk.ts
    └── utils.ts
```

React Renderer

```
src/
├── pages/          # 12 routes
├── features/       # 8 feature modules
├── components/     # 48 shared
├── hooks/          # 27 custom hooks
├── stores/         # 6 Zustand stores
├── lib/            # Core utilities
│   └── voice/      # ElevenLabs
├── config/         # Centralized config
└── types/          # TypeScript types
```

03 Electron Main Process

3.1 Window Lifecycle

The main window is created with hardened defaults:

```
{
  contextIsolation: true,    // Strict renderer isolation
  sandbox: true,             // OS-level sandboxing
  nodeIntegration: false,    // No Node.js in renderer
  webSecurity: true,         // Enforce same-origin
  devTools: !isProduction    // Disabled in release builds
}
```

Window dimensions default to 1400×900 with a minimum of 1000×700. On macOS, the title bar is hidden with custom traffic light positioning for a native feel. The window is shown only after the `ready-to-show` event to prevent visual flash.

3.2 Navigation Guards

All navigation events are intercepted and validated:

- `will-navigate` — Only allows navigation to the renderer origin (`file://` or `localhost` dev server). All other URLs are blocked.
- `will-redirect` — Same validation applied to server-initiated redirects.
- **Window open handler** — External HTTPS links matching the URL allowlist open in the system browser via `shell.openExternal()`. Everything else is denied.
- **Webview prevention** — The `web-contents-created` handler blocks `<webview>` tag attachment entirely.

3.3 System Tray Integration

A persistent system tray icon provides always-available agent status:

- Context menu with Open, Agent Status, Check for Updates, and Quit actions
- Dynamic tooltip reflecting running agent count (e.g., “2 agents running”)
- Double-click restores the main window
- Configurable minimize-to-tray behavior with a one-time instructional notification
- Agent status updates received from the renderer via IPC

3.4 Auto-Update Pipeline

Updates are managed via `electron-updater` with GitHub Releases as the provider:

01	02	03	04	05
Idle	Checking	Available	Downloading	Ready

The updater checks on startup (after a 5-second delay) and then hourly. Updates auto-install on application quit. Desktop notifications alert the operator when an update is available or ready. The updater is disabled during E2E tests, in development, and on Linux without AppImage.

3.5 IPC Handler Architecture

The main process registers 22 validated IPC handlers organized by domain:

DOMAIN	HANDLERS
Store	`get`, `set`, `delete` (22-key allowlist)
OAuth	Shopify, Gorgias, Zendesk authorization flows
Auth	`setApiKey`, `getApiKey`, `hasApiKey`, `clearApiKey`
Secrets	Secure credential storage via `safeStorage`
Window	Minimize, maximize, close, full-screen
Notifications	System notification dispatch
App	Version, platform info, dev tools
Background	Agent status reporting to tray
Updates	Check, download, install triggers

All store operations validate keys against a static allowlist. API key inputs are capped at 4,096 characters. Secret values are limited to 10MB with serialization verification.

04 Renderer Application

4.1 Routing & Page Structure

The application uses React Router v6 with lazy-loaded page components:

ROUTE	COMPONENT	PURPOSE
`/`	Dashboard	Agent session list, stats, bulk operations
`/agent/:sessionId`	AgentConsole	Real-time streaming agent interaction
`/playground`	ChatPlayground	Interactive agent chat testing
`/voice`	Voice	Voice agent interface (ElevenLabs STT/TTS)
`/analytics`	Analytics	Usage metrics and performance charts
`/connections`	Connections	Platform OAuth integrations and MCP servers
`/templates`	Templates	Reusable agent configuration templates
`/webhooks`	Webhooks	Webhook endpoint management
`/settings`	Settings	Application preferences (7 subsections)
`/audit-log`	AuditLog	System activity trail
`/login`	Login	API key authentication
`/register`	Register	Account registration

All pages are wrapped in a `ProtectedRoute` guard that validates authentication before rendering. A `Suspense` boundary with a `PageLoader` fallback handles chunk loading.

4.2 Layout Architecture

- **Sidebar** (264px) — Logo, brand selector dropdown, 9 navigation items with Lucide icons, user section with logout and version display, Electron-native drag region
- **Top Bar** — Workspace name, page title, API health indicator, notification bell with unread count, theme toggle, command palette trigger
- **Content Area** — Scrollable page content with animated route transitions via Framer Motion

4.3 Command Palette

A global command palette (`Cmd/Ctrl+K`) provides instant access to:

- Page navigation (all routes searchable)
- Active agent sessions (searchable by name/ID)
- Quick actions (create agent, refresh data, toggle theme)

The palette uses fuzzy matching, keyboard arrow navigation, and renders in a modal overlay.

4.4 Theming System

The theming system uses CSS custom properties controlled by `data-*` attributes on the document root. Six accent colors and two base themes produce 12 visual combinations. The `data-reduce-motion` attribute respects accessibility preferences by collapsing all transitions.

> **IMPLEMENTATION NOTE**
>
> Tailwind utilities reference brand colors via CSS variable indirection (`text-brand-500` , `bg-brand-600`), using `color-mix()` for opacity modifiers since `@apply` with CSS variable-based color opacity fails in Tailwind.

4.5 Accessibility

- **Semantic HTML** — Proper use of `<nav>`, `<main>`, `<aside>`, ARIA roles and labels
- **Skip link** — "Skip to content" link for keyboard users
- **Focus management** — `useFocusTrap()` for modals, `useAutoFocus()` for form fields
- **Keyboard navigation** — Full list navigation with arrow keys, Home/End, multi-select via Shift/Ctrl
- **Screen reader support** — ARIA attributes on all interactive elements
- **Reduced motion** — Respects `prefers-reduced-motion` and user preference
- **E2E testing** — Dedicated Playwright accessibility suite using axe-core

4.6 Keyboard Shortcuts

SHORTCUT	ACTION
`Cmd/Ctrl+K`	Open command palette
`Cmd/Ctrl+F`	Search messages (Agent Console)
`Cmd/Ctrl+E`	Export conversation
`Cmd/Ctrl+Shift+L`	Toggle logs panel
`Escape`	Close modals, dismiss search
Arrow keys	Navigate lists, command palette
`Enter/Space`	Activate selected item

05

Data Model & Type System

5.1 Core Domain Types

The application is built around a strongly-typed domain model defined in `src/types/index.ts`:

Agent Session

```
interface AgentSession {
  id: string;
  tenant_id: string;
  brand_id: string;
  agent_type: string;
  name?: string | null;
  tags?: string[] | null;
  status: AgentSessionStatus;
  // starting | running | paused | stopping | stopped | failed
  config: AgentSessionConfig;
  metrics: AgentSessionMetrics;
  created_at: string;
  updated_at: string;
}
```

Agent Configuration

```
interface AgentSessionConfig {
  loop_interval_ms: number;       // ≤100ms = manual mode
  max_iterations: number;         // Safety cap on loop count
  iteration_timeout_secs: number; // Per-iteration timeout
  pause_on_error: boolean;        // Auto-pause on failure
  custom_instructions?: string;   // Operator system prompt
  mcp_servers: string[] | null;  // Model Context Protocol
  model: string;                  // LLM model identifier
  temperature: number;            // Sampling temperature
}
```

5.2 Multi-Tenant Hierarchy

```
Tenant (organization)
  └── Brand (customer-facing entity)
        ├── Agent Sessions (running AI agents)
        ├── Platform Connections (Shopify, Gorgias, Zendesk, ...)
        └── Webhooks (event subscriptions)
```

Each tenant contains one or more brands. Brands isolate agent sessions, platform credentials, and webhook configurations. Operators switch brands via a sidebar dropdown, and the selection is persisted to preferences.

5.3 Event Types (SSE)

The streaming protocol defines nine event types:

EVENT	KEY FIELDS	PURPOSE
`status_changed`	status, message	Agent state transitions
`thinking`	content	Agent reasoning visibility
`message`	id, role, content	User/assistant messages
`tool_call`	id, tool_name, arguments	Tool invocation
`tool_result`	tool_call_id, result, success	Tool execution output
`log`	level, message, metadata	Operational logging
`error`	code, message, recoverable	Error reporting
`metrics`	loop_count, tokens_used	Performance counters
`heartbeat`	timestamp	Connection keepalive

5.4 Schema Validation

All API responses are validated at the boundary using Zod schemas (`src/lib/schemas.ts`) before entering the application. Schemas use `.passthrough()` for forward compatibility — unknown fields from newer API versions are preserved, not rejected.

> **DESIGN DECISION**
>
> Built-in transforms handle data normalization (e.g., computing webhook delivery `success` from `response_status`, stringifying nested payloads). The `unwrapDataEnvelope()` utility normalizes the engine's `{ok, data: T}` wrapper format.

06

API Client & Resilience Layer

> *A four-layer defense — deduplication, retry, circuit breaker, offline cache – ensures operators maintain visibility even during API degradation.*

6.1 Request Pipeline

```
Caller → apiRequest<T>()
           |
           ├─ Request Deduplication (GET only, 100ms TTL)
           |    └─ Cache hit? Return shared Promise
           |
           ├─ Circuit Breaker Gate
           |    └─ OPEN? Reject immediately
           |
           ├─ Auth Header Construction
           |    └─ Try ApiKey → Bearer → X-API-Key formats
           |
           ├─ Retry Loop (up to 3 attempts)
           |    ├─ fetch() with AbortController timeout (15s)
           |    ├─ Exponential backoff: 1s × 2^attempt + jitter (25%)
           |    ├─ Re-check circuit breaker before retry
           |    └─ Only retry on: 429, 500, 502, 503, 504
           |
           ├─ Circuit Breaker Update
           |    └─ onSuccess() or onFailure()
           |
           ├─ Zod Schema Validation
           |    └─ validateResponse(schema, data)
           |
           └─ Metrics Recording
                └─ Status, duration, retries, cache hit
```

6.2 Configuration

```
const API_CONFIG = {
  baseUrl: 'https://engine.stateset.cloud.stateset.app',
  timeout: 15000,
  maxRetries: 3,
  retryableStatusCodes: [429, 500, 502, 503, 504],
  circuitBreaker: {
    maxFailures: 5,
    halfOpenTimeout: 30000,
    resetTimeout: 60000,
  },
  stream: {
    maxEvents: 500,
    maxMessages: 500,
    maxReconnectAttempts: 12,
    reconnectBaseDelay: 1000,
    reconnectMaxDelay: 30000,
  },
};
```

6.3 Circuit Breaker

```
          5 consecutive failures
CLOSED ──────────────────────────► OPEN
  ▲                                  │
  │ 2 successes                      │ 30s timeout
  │                                  ▼
  └──────────────── HALF_OPEN ◄──────┘
                       │
                  any failure
                       │
                       └──────────► OPEN
```

Configuration: 5 failures to open, 30-second half-open timeout, 60-second failure count reset. State changes are observable via callbacks for UI integration (the `ApiHealthIndicator` component renders a colored dot reflecting circuit state).

6.4 Request Deduplication

Concurrent identical GET requests share a single in-flight promise. The deduplicator uses a `method:path:params` cache key with a 100ms TTL and a maximum of 100 entries. This prevents redundant network calls during rapid component remounting or React Query background refetches.

6.5 Fallback Endpoint Resolution

For API endpoints undergoing versioning or path migration, the client supports multi-path fallback. Each path is attempted in order; the first successful response is used. This provides forward compatibility during API evolution.

```
fetchWithFallbackPaths([
  `/api/v1/tenants/${tenant}/brands/${brand}/agents`,
  `/api/v1/tenants/${tenant}/agents`,
  `/api/v1/agents`,
]);
```

6.6 Auth Header Strategy

The API client supports multiple authentication header formats:

1. **ApiKey** — `Authorization: ApiKey <key>` (primary)
2. **Bearer** — `Authorization: Bearer <key>` (fallback)
3. **X-API-Key** — Custom header (legacy support)

07 Real-Time Streaming

7.1 SSE Architecture

Agent sessions emit real-time events via Server-Sent Events. The `useAgentStream()` hook manages the full lifecycle:

```
Agent Console mounts
  │
  ├─ useAgentStream(sessionId) initializes
  │    ├─ Selects auth method (stream token → API key header → query param)
  │    ├─ Opens EventSource to /api/v1/.../agents/:id/stream
  │    └─ Registers event handlers for all 9 event types
  │
  ├─ Events arrive
  │    ├─ Parsed → deduplicated by event ID
  │    ├─ Status changes → update session status
  │    ├─ Messages → append to message list (capped at 500)
  │    ├─ Metrics → update counters (loop_count, tokens, cost)
  │    ├─ Tool calls/results → render in message timeline
  │    └─ Heartbeats → reset connection timeout
  │
  ├─ Connection lost
  │    ├─ Exponential backoff: 1s × 2^attempt + 25% jitter
  │    ├─ Max 12 reconnection attempts
  │    └─ Immediate reconnect on navigator.onLine change
  │
  └─ Agent Console unmounts → close EventSource, clear timers
```

7.2 Connection Management

- **Auth cascade** — Tries stream token, then API key header, then API key query parameter
- **Exponential backoff** — 1s base, 2× multiplier, 30s ceiling, 25% jitter
- **Max reconnects** — 12 attempts before giving up
- **Online detection** — Triggers immediate reconnect on `navigator.onLine` change
- **Buffer management** — Events capped at 500, messages at 500 to bound memory
- **Typing indicator** — Auto-cleared after 30 seconds of inactivity
- **Deduplication** — Events deduplicated by ID to handle reconnection replays

7.3 Manual vs. Autonomous Mode

Autonomous Mode

`loop_interval_ms > 100` — Agent iterates automatically at the configured interval. The console displays a continuous stream of events.

Manual Mode

`loop_interval_ms ≤ 100` — Agent pauses after each iteration, waiting for operator input. The operator drives the conversation turn-by-turn.

7.4 Agent Console UI

- **Message stream** — Scrollable message list with auto-scroll (500px threshold) and manual scroll lock
- **Message search** — `Cmd/Ctrl+F` to search within the message history
- **Tool call visualization** — Expandable tool call/result pairs with syntax highlighting
- **Metrics panel** — Live counters for tokens, tool calls, loop iterations, cost, and uptime
- **Configuration modal** — Runtime agent parameter adjustment
- **Agent toolbar** — Status badge, start/pause/stop controls, export, clone, log replay
- **Approval dialog** — Human-in-the-loop approval for sensitive agent actions

08 State Management

8.1 Dual-Layer Architecture

Server State (React Query)

- Agent sessions, brands, webhooks, connections, analytics
- Automatic background refetch on window focus
- Configurable polling intervals (5s default for sessions)
- Cache invalidation tied to mutations
- Query key factory ensures consistent cache addressing

Client State (Zustand)

STORE	PERSISTENCE
`auth.ts`	OS keychain
`preferences.ts`	electron-store
`ui.ts`	In-memory
`notifications.ts`	electron-store
`auditLog.ts`	electron-store
`templates.ts`	electron-store

8.3 Optimistic Mutations

Session lifecycle operations use optimistic updates for immediate UI feedback:

01	02	03	04	05
Cancel pending queries	Snapshot cache	Apply optimistic update	Execute mutation	Invalidate or rollback

This pattern is encapsulated in the `useOptimisticSessionMutation()` hook, which handles cancellation, snapshot, rollback, and cache invalidation as a single unit.

8.4 Persistence Strategy

STORE	BACKEND	ENCRYPTION
API keys	Electron `safeStorage`	OS keychain (Keychain/Credential Manager)
Preferences	`electron-store`	Optional (AES with env key)
Audit log	`electron-store`	Optional
Custom templates	`electron-store`	Optional
Offline cache	IndexedDB	None (non-sensitive data)
Chat history	LocalStorage	None
Voice settings	LocalStorage	None
Session UI state	In-memory	N/A

09

Security Architecture

> *Process isolation, CSP enforcement, IPC allowlisting, and OS-level credentia encryption establish a zero-trust boundary between the network and the operator.*

9.1 Defense in Depth

Process Level

- Context isolation between main and renderer
- Sandboxed renderer with no Node.js access
- Single-instance lock prevents duplicate processes

Network Level

- Content Security Policy restricting script, connect, and frame sources
- Security headers: `X-Frame-Options: DENY`, `X-Content-Type-Options: nosniff`, `Referrer-Policy: strict-origin-when-cross-origin`
- URL allowlist for external navigation
- Protocol blocking: `javascript:`, `data:`, `vbscript:`, `blob:` all denied

IPC Level

- No raw `ipcRenderer` exposed to renderer
- All IPC methods are explicitly typed in the preload bridge
- Store key allowlist prevents arbitrary data access
- Input validation: string length limits, integer bounds checking, serialization verification

Credential Level

- API keys encrypted via OS `safeStorage` API
- Migration path from legacy unencrypted storage
- CSRF tokens (random 16-byte hex) for OAuth flows
- Timing-safe HMAC comparison for Shopify webhooks

Data Level

- Sentry events sanitized before transmission — redacts API keys, tokens, JWTs, emails, AWS credentials
- Multi-pass redaction with intermediate placeholders prevents partial exposure
- Audit log records all significant operator actions

9.3 Content Security Policy

```
default-src 'self';
script-src 'self';
style-src 'self' https://fonts.googleapis.com 'unsafe-inline';
font-src 'self' https://fonts.gstatic.com data:;
connect-src 'self' <configured-api-hosts> wss://api.elevenlabs.io;
media-src 'self' blob: data: https:;
object-src 'none';
frame-ancestors 'none';
base-uri 'none';
form-action 'self';
upgrade-insecure-requests;
```

> **PRAGMATIC CONCESSION**
>
> `unsafe-inline` for styles is required for Tailwind CSS compatibility, mitigated by the absence of user-generated style content.

10 Offline Resilience & Caching

10.1 Offline Cache (IndexedDB)

The application maintains an IndexedDB cache for critical entity types: sessions, brands, connections, and auth context. Each cache entry carries a TTL.

```
Query fires → Network available?
  ├─ Yes → Fetch from API, cache result
  └─ No  → Return cached data, show offline banner
```

10.2 Auth Resilience

1. Attempt to validate stored API key against `/api/v1/auth/me`
2. If network error → Accept cached credentials and allow offline operation
3. If 401/403 → Clear stored key, redirect to login
4. If 5xx → Accept provided auth data as fallback

10.3 Health Monitoring

The `useOnlineStatus()` hook provides multi-dimensional health assessment:

- **Connectivity** — `navigator.onLine` + periodic health endpoint checks
- **API reachability** — Latency measurement against `/health`
- **Component health** — Database, NATS, Redis status from detailed health endpoint
- **Circuit breaker state** — Both server-side and client-side circuit states
- **Retry logic** — Exponential backoff for health checks (30s base, 2min ceiling)

11 Platform Integrations & OAuth

11.1 Supported Platforms

PLATFORM	AUTH METHOD	KEY CAPABILITIES
Shopify	OAuth 2.0	Orders, customers, products, fulfillments
Gorgias	OAuth 2.0	Tickets, customers, account info
Zendesk	OAuth 2.0	Tickets, users, read/write access
Recharge	API Key	Subscription management
Klaviyo	API Key	Email/SMS marketing automation
Make	API Key	Workflow automation
N8N	API Key	Workflow automation
Zapier	API Key	Integration automation
Custom MCP	Manual	Arbitrary Model Context Protocol servers

11.2 OAuth 2.0 Flow

01	02	03	04	05
Start local HTTP server	Generate CSRF token	Open OAuth window	User authorizes	Validate & exchange code

SECURITY MEASURES

State parameter validation prevents CSRF attacks. Authorization code length capped at 2,048 characters. Domain validation for provider subdomains. Timing-safe HMAC-SHA256 verification for Shopify callbacks. 5-minute timeout on OAuth windows. Client credentials auto-cleared in production.

11.3 Credential Storage Modes

Local

Encrypted via `safeStorage` on the operator's machine. Credentials never leave the device.

Vault

Stored server-side via the StateSet Engine API's secrets endpoint. Enables credential sharing across multiple desktop instances.

12 Voice Interface

12.1 Architecture

```
Operator speaks → Browser MediaRecorder API
  → Audio blob → ElevenLabs STT API
  → Transcription text → StateSet Engine API (agent chat)
  → Agent response text → ElevenLabs TTS API
  → Audio playback → Browser Audio API
```

12.2–12.3 Speech Pipeline

Speech-to-Text

- ElevenLabs STT API with configurable model selection
- Browser `MediaRecorder` for audio capture
- Visual recording indicator with waveform animation
- Auto-send transcription on recording stop

Text-to-Speech

- ElevenLabs TTS API with configurable voice selection
- Auto-speak toggle for hands-free operation
- Barge-in support — interrupt by starting a new recording
- Voice settings persisted to LocalStorage

12.4 Conversation Focus

SUPPORT

Customer service, ticket triage, escalation

OPERATIONS

Order management, inventory, fulfillment

GROWTH

Marketing, retention, customer engagement

12.5–12.6 Response Controls

Operators can control response verbosity across three levels: **Concise** (brief, actionable), **Balanced** (standard detail), and **Detailed** (comprehensive analysis). The voice interface maintains the last 40 messages in memory, providing context continuity without unbounded memory growth.

13 Agent Templates & Configuration

13.1 Template System

```
interface AgentTemplate {
  id: string;
  name: string;
  description: string;
  icon: string;
  color: string;
  category: 'general' | 'support' | 'commerce' | 'automation' | 'custom';
  agentType: string;
  config: Partial<AgentSessionConfig>;
  isCustom?: boolean;
}
```

13.2 Built-in Templates

CUSTOMER SUPPORT AGENT

Ticket triage, response drafting, escalation routing

E-COMMERCE AGENT

Order lookup, return processing, inventory queries

AUTOMATION AGENT

Workflow execution, data synchronization, webhook handling

13.4 Agent Creation Flow

```
Dashboard → "New Agent" button → CreateAgentDialog
  ├─ Select template (or start from scratch)
  ├─ Configure: name, model, temperature, instructions
  ├─ Optional: attach MCP servers
  ├─ Optional: set sandbox API key for isolated execution
  └─ Submit → POST /api/v1/.../agents → New session created
```

14 Observability & Diagnostics

14.1 Structured Logging

- **JSON output** in production, human-readable format in development
- **Session correlation** — Unique session ID in all log entries
- **Scoped loggers** — `apiLogger`, `authLogger`, `agentLogger`
- **Sensitive data masking** — API keys, tokens, passwords redacted
- **Performance timing** — `log.time()` for measuring async operations
- **Buffer** — Last 1,000 entries retained for export and debugging

14.2 Request Metrics

The metrics module maintains a rolling window of the last 500 API calls and computes:

p50 LATENCY PERCENTILE	p95 LATENCY PERCENTILE	p99 LATENCY PERCENTILE
% ERROR RATE	% CACHE HIT RATE	# CIRCUIT BREAKER TRIPS

14.3 Analytics Dashboard

METRIC CARD	DESCRIPTION
Total Agents	Count of all agent sessions
Running Agents	Currently active sessions
Total Tokens	Aggregate token consumption
Total Tool Calls	Aggregate tool invocations
Total Loops	Aggregate iteration count
Total Errors	Aggregate error count
Avg Tokens/Agent	Mean token usage per session
Avg Tool Calls/Agent	Mean tool calls per session

14.6 Error Classification

CATEGORY	RETRYABLE	SUGGESTED ACTION
`NETWORK`	Yes	“Try Again”
`SERVER`	Yes	“Try Again”
`RATE_LIMIT`	Yes	“Try Again” (after delay)
`AUTH`	No	“Log In”
`VALIDATION`	No	“Dismiss”
`CLIENT`	No	“Dismiss”
`UNKNOWN`	No	“Contact Support”

15 Testing Strategy

15.1 Test Pyramid

```
          ┌─────────┐
          │  E2E    │  Playwright (4 projects)
          │  Tests  │  - Electron lifecycle
         ┌└─────────┘┐ - Visual regression
         │ Integration │ - Accessibility (axe-core)
         │   Tests     │ - Cross-feature flows
        ┌└─────────────┘┐
        │  Component     │  Vitest + Testing Library
        │    Tests       │  - 78 test files
       ┌└────────────────┘┐ - happy-dom environment
       │   Unit Tests      │ - API, stores, hooks, utilities
       └───────────────────┘
```

15.2 Unit & Component Testing

LAYER	FILES	SCOPE
Libraries	21	API client, schemas, circuit breaker, metrics, cache, errors
Components	15	Modal, Button, Dropdown, Pagination, ErrorBoundary
Pages	11	Login, Dashboard, AgentConsole, Settings, Analytics
Hooks	10	useErrorHandler, useOptimisticSessionMutation, useAgentStream
Stores	6	auth, preferences, notifications, auditLog, ui, templates
Features	8	agent-console, webhooks, chat-playground, connections

15.3 End-to-End Testing

PROJECT	PURPOSE
`electron`	App lifecycle, IPC communication, window management
`visual`	Screenshot comparison with 2% diff tolerance
`accessibility`	WCAG compliance via axe-core
`integration`	Cross-feature user flows

15.4 Code Quality Gates

- **ESLint** — Zero-warning policy with TypeScript and React hooks rules
- **Prettier** — Consistent formatting (single quotes, trailing commas, 100-char width)
- **TypeScript** — Strict mode with `noEmit` type checking
- **Pre-commit hooks** — Husky + lint-staged runs ESLint fix and Prettier on staged files
- **CI pipeline** — Lint → Typecheck → Unit tests → E2E → Build (all must pass)

16 Build & Distribution

16.1 Build Pipeline

01	02	03	04
Test (Lint, Types, Unit)	E2E (xvfb)	Build (macOS, Win, Linux)	Release (GitHub)

16.2 Platform Artifacts

PLATFORM	FORMATS	CODE SIGNING
macOS (Intel + Apple Silicon)	DMG, ZIP	Hardened runtime, team signing
Windows (x64, arm64)	NSIS installer, portable	SHA-256 certificate
Linux (x64)	AppImage, .deb	N/A

16.4 Vite Code Splitting

CHUNK	CONTENTS
`react-vendor`	React, React DOM, React Router
`ui-vendor`	Framer Motion, Lucide icons
`data-vendor`	React Query, Zustand, date-fns

16.5 Runtime Requirements

≥22.12 NODE.JS	35.7.5 ELECTRON	65 RUNTIME DEPS
61 DEV DEPS		

17 Future Architecture Considerations

While the current architecture is production-hardened, several areas present opportunities for further improvement as the platform scales.

17.1 Main-Process SSE Streaming

> **PROBLEM**
>
> The SSE streaming connection lives in the Renderer process. Chromium aggressively throttles background renderers — timers, network streams, and `requestAnimationFrame` are deprioritized when a window is hidden or minimized.

Recommendation: Migrate the SSE connection layer to the Node.js Main Process, which is never subject to Chromium's background throttling. The main process would maintain persistent SSE connections, buffer events, and forward rate-limited updates to the renderer via IPC when active.

17.2 Custom Protocol Handler for OAuth

> **PROBLEM**
>
> OAuth flows rely on fixed local HTTP servers bound to ports 8234–8236. In enterprise environments, these ports may be blocked by corporate firewalls or VPN clients.

Recommendation: Register a custom protocol handler via `app.setAsDefaultProtocolClient('stateset')` and use `stateset://oauth/callback` as the redirect URI. This eliminates port conflicts entirely.

17.3 Event Stream Memory Management

> **PROBLEM**
>
> The SSE event buffer is capped at 500 events. For long-running sessions, holding hundreds of deeply nested JSON objects creates GC pressure and potential UI stutter.

Recommendation: Implement a tiered storage strategy: keep only the most recent 200 events in React state, persist the full history to IndexedDB or SQLite, and pair with virtual scrolling for on-demand rendering.

17.4 Integrated Query Persistence

Recommendation: Replace the custom IndexedDB wrapper with TanStack Query's official `PersistQueryClient` plugin backed by an IndexedDB persister. This eliminates the dual-cache layer and unifies invalidation logic.

17.5 Local AI Fallback for Offline Operation

Recommendation: Integrate a lightweight local LLM (via `Transformers.js`, Ollama, or `llama.cpp`) to enable offline capabilities: drafting customer replies, summarizing session history, or generating configuration suggestions. Actions taken offline would be queued and synced when connectivity is restored.

17.6 Multi-Operator Conflict Resolution

Recommendation: For collaborative resources, integrate a CRDT library such as Yjs or Automerge for true multiplayer editing. For simpler cases, server-side optimistic locking with version vectors (ETags) provides conflict detection.

17.7 Voice Streaming Optimization

Recommendation: Adopt WebSocket-based streaming for both STT and TTS. The STT stream can begin transcription while the operator is still speaking, and the TTS stream can begin playback as first tokens arrive — creating sub-second perceived latency.

18 Conclusion

StateSet Desktop demonstrates that desktop applications remain the right choice for operational tooling where reliability, security, and system integration are paramount. The architecture achieves its design goals through:

Resilience — A four-layer defense (deduplication → retry → circuit breaker → offline cache) ensures operators maintain visibility even during API degradation.

Security — Process isolation, CSP enforcement, IPC allowlisting, and OS-level credential encryption establish a zero-trust boundary between the network and the operator.

Performance — Lazy-loaded routes, virtual scrolling, request deduplication, and optimistic mutations keep the interface responsive under load.

Observability — Structured logging, percentile metrics, circuit breaker visibility, and audit trails make the system transparent to both operators and developers.

Quality — 78 test files, four E2E test projects, strict linting, and type checking enforce correctness across the codebase.

Multimodal Interaction — Text-based agent consoles, voice interfaces with STT/TTS, and a chat playground provide operators with multiple modalities for agent interaction.

> *The application serves as infrastructure for autonomous agent operations – control plane where reliability is not a feature but a requirement.*

StateSet

StateSet, Inc. · MIT License · Version 1.1.15 · March 2026